

# Clase 326 — Análisis de malware para respuesta a incidentes

---

Parte: **17 — Profundización para certificaciones** · Fuente: *Practical Malware Analysis (Sikorski & Honig)* · **SANS FOR610** 🏆 Duración estimada: **150 min** · Nivel: **Experto**

---

## **Objetivo**

---

Aprender el análisis de malware **orientado a la respuesta a incidentes**: no un desensamblado exhaustivo, sino un **triaje rápido** que en minutos extrae los IOCs necesarios para contener y erradicar la amenaza, alimenta la línea de tiempo del incidente y produce un informe DFIR accionable. El foco es la velocidad y la utilidad operativa que exigen los roles de analista de incidentes de **BTL1** y **SANS FOR610**, no la ingeniería inversa completa.

## **Resultados de aprendizaje**

---

Al finalizar, el alumno podrá:

1. **Ejecutar** un triaje rápido de un binario sospechoso (hash, tipo, firmas, capacidades) sin detonarlo.
2. **Extraer IOCs** de red, host y comportamiento (dominios, IPs, mutex, claves de registro, hashes).
3. **Detonar** de forma controlada una muestra en sandbox aislada y capturar su comportamiento.
4. **Aplicar** reglas YARA y `capa` para clasificar la familia y las capacidades del malware.
5. **Correlacionar** los IOCs y las marcas de tiempo con la línea de tiempo del incidente.
6. **Redactar** un informe DFIR con resumen ejecutivo, IOCs, TTPs (mapeados a ATT&CK) y recomendaciones.

## Temas

| # | Tema                                                                           | Por qué importa                                                    |
|---|--------------------------------------------------------------------------------|--------------------------------------------------------------------|
| 1 | Triaje estático rápido (hash, <code>file</code> , PE headers)                  | Decide en segundos si vale la pena un análisis profundo            |
| 2 | Extracción de cadenas e imports ( <code>strings</code> , <code>pefile</code> ) | Revela URLs, mutex y APIs que delatan la intención                 |
| 3 | Clasificación con YARA y <code>capa</code>                                     | Identifica familia y capacidades sin ejecutar el binario           |
| 4 | Análisis dinámico controlado (sandbox)                                         | El comportamiento en ejecución confirma lo que el estático sugiere |
| 5 | Extracción de IOCs de red y host                                               | Insumo directo para contención (bloqueo, aislamiento)              |
| 6 | Mapeo a MITRE ATT&CK                                                           | Traduce comportamiento en TTPs comunicables                        |
| 7 | Correlación con la timeline del incidente                                      | Ubica la muestra en la secuencia de compromiso                     |
| 8 | Informe DFIR                                                                   | Convierte el análisis en acción para el equipo y la dirección      |

## Definiciones y características

- **Triaje de malware:** evaluación inicial rápida para priorizar. Característica clave: busca *suficiente* información para decidir y actuar, no el análisis total.
- **IOC (Indicator of Compromise):** artefacto observable que evidencia una intrusión (hash, IP, dominio, mutex, ruta). Característica clave: accionable para detección y bloqueo.
- **Análisis estático:** examen del binario sin ejecutarlo. Característica clave: seguro y rápido, pero limitado frente a ofuscación y empaquetado.
- **Análisis dinámico:** observación del malware en ejecución en un entorno controlado. Característica clave: revela comportamiento real, pero requiere aislamiento estricto.
- **YARA:** lenguaje de reglas para identificar patrones en archivos/memoria. Característica clave: reutilizable para *hunting* a escala.
- **capa** : herramienta que infiere capacidades del binario (p. ej. "crea servicio", "captura teclado") a partir de patrones. Característica clave: traduce bytes en comportamiento legible y lo mapea a ATT&CK.
- **TTP (Tácticas, Técnicas y Procedimientos):** el *cómo* del adversario. Característica clave: más estable que los IOCs (la "pirámide del dolor" de David Bianco).
- **Empaquetador (packer):** comprime/cifra el binario para dificultar el análisis. Característica clave: alta entropía y pocos imports son señales típicas.

## Herramientas y preparación

 **Recordatorio de laboratorio aislado.** El análisis dinámico **detona malware real**. Hazlo únicamente en una VM desechable, con **snapshot** previo, red en modo *host-only* o simulada (INetSim/FakeNet-NG), sin carpetas compartidas ni credenciales reales, y sin acceso a la red

corporativa. Nunca detones una muestra en tu equipo de trabajo ni en una VM con salida a Internet sin control. Restaura el snapshot tras cada ejecución.

- **Estático:** `file`, `sha256sum`, `strings`, `pefile` / PE-bear, Detect It Easy (DIE), `capa`, `yara`.
- **Dinámico:** VM Windows aislada + Procmon, Process Hacker, Regshark/Regshot, Wireshark, INetSim o FakeNet-NG para simular servicios de red.
- **Sandbox automatizada (opcional):** CAPEv2 o Cuckoo local; o servicios en línea (Any.Run, Hybrid Analysis, VirusTotal) **solo si la muestra no es confidencial**, ya que subirla la hace pública.
- **Muestras seguras de práctica:** repositorios educativos con *defanged samples*, retos de MalwareBazaar/CyberDefenders o binarios EICAR para validar el flujo sin riesgo real.

## Laboratorio guiado

Analizaremos `muestra.exe` en dos fases: estática (host de análisis) y dinámica (VM aislada con snapshot).

### Fase 1 — Triage estático (sin ejecutar):

1. **Identifica y hashea** la muestra:

```
bash file muestra.exe sha256sum muestra.exe | tee muestra.sha256
```

Busca el hash en tu inteligencia de amenazas para ver si ya está catalogado. 2. **Mide la entropía y detecta empaquetado:**

```
bash die muestra.exe # o: python3 -c "import pefile; ... "
```

Secciones con entropía > 7.0 e imports casi vacíos sugieren *packer*. 3. **Extrae cadenas e imports relevantes:**

```
bash strings -n 8 muestra.exe | grep -Ei 'http|\.exe|\.dll|CreateService|reg add|mutex'
```

Anota URLs, nombres de mutex, rutas y APIs (p. ej. `WinExec`, `URLDownloadToFile`). 4. **Infiere capacidades y familia:**

```
bash capa -v muestra.exe # capacidades mapeadas a ATT&CK yara reglas.yar muestra.exe #  
coincidencia de familia
```

### Fase 2 — Análisis dinámico controlado (VM aislada, snapshot tomado):

5. **Prepara la captura de comportamiento:** inicia Procmon (filtrado por el nombre del proceso), Wireshark y toma un Regshot "antes".
6. **Simula la red** para que el malware "hable" sin salir a Internet:

```
text INetSim (o FakeNet-NG) # responde DNS/HTTP/HTTPS de forma controlada
```

7. **Detona la muestra** y déjala correr un par de minutos observando Procmon y Process Hacker (procesos hijos, inyección, persistencia).
8. **Captura los IOCs de host:** toma Regshot "después" y compáralo; registra claves de persistencia (`Run`, servicios, tareas programadas), archivos creados y mutex.
9. **Captura los IOCs de red:** en Wireshark/INetSim anota dominios de C2, IPs, User-Agent y patrones de *beacon*.

10. **Restaura el snapshot** de la VM para dejarla limpia antes de cualquier otra prueba.

### Fase 3 — Correlación e informe:

11. **Mapea el comportamiento a MITRE ATT&CK** (p. ej. persistencia T1547, C2 T1071) y arma la tabla de TTPs.
12. **Correlaciona** las marcas de tiempo de creación de archivos/registro con la timeline del incidente (logs de EDR/Sysmon) para ubicar el momento de la infección.
13. **Redacta el informe DFIR** con las secciones del apartado siguiente.

## Ejercicios

---

1. Toma tres muestras de práctica y clasifícalas como *empaquetada* / *no empaquetada* usando entropía e imports.
2. Extrae de una muestra al menos cinco IOCs (2 de red, 3 de host) y preséntalos en formato tabla.
3. Escribe una regla YARA sencilla que detecte una cadena única de tu muestra y válícala.
4. Detona una muestra en la VM aislada y documenta su mecanismo de persistencia con evidencia de Regshot.
5. Ejecuta `capa` sobre un binario y traduce tres capacidades a técnicas ATT&CK con su ID.
6. Correlaciona la hora de un archivo soltado por el malware con un evento de Sysmon para situarlo en la timeline.

## Reto verificable

---

Recibes `dropper.exe` como parte de un incidente en curso. **Criterio de aceptación:** entregas un mini-informe DFIR de una página que incluye (a) hash SHA-256 y veredicto de empaquetado, (b) al menos 4 IOCs accionables (dominio/IP de C2, mutex, clave de persistencia), (c) 3 técnicas ATT&CK con su ID justificadas por el comportamiento observado, y (d) una recomendación de contención concreta (qué bloquear/aislar). El reto se logra si un analista de contención puede actuar directamente con tus IOCs sin volver a analizar la muestra.

## ⚠ Errores comunes

| Síntoma / mensaje                                                | Causa y cómo arreglar                                                                                             |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| La muestra no hace "nada" al detonarla                           | Detecta la VM o espera comando C2; da salida de red simulada (INetSim) y revisa evasión anti-VM                   |
| <code>strings</code> no muestra URLs ni imports útiles           | Binario empaquetado; desempácalo (unpacking) o vuelca la memoria tras ejecución y aplica <code>strings</code> ahí |
| Subiste la muestra a VirusTotal y era confidencial               | La subida la hace pública; para muestras sensibles usa solo búsqueda por hash o sandbox local                     |
| La VM de análisis "se infectó de verdad" y afectó otras máquinas | Faltó aislamiento de red o snapshot; usa red host-only/simulada y restaura snapshots siempre                      |
| IOCs demasiado frágiles (solo un hash)                           | Un hash cambia con recompilar; prioriza IOCs de comportamiento y TTPs (pirámide del dolor)                        |
| El informe es técnico pero nadie actúa                           | Falta resumen ejecutivo y recomendaciones claras; separa audiencia técnica de la directiva                        |

## ? Preguntas frecuentes

? **¿Cuánto análisis es suficiente en un incidente?** El justo para contener y erradicar: IOCs accionables, mecanismo de persistencia y TTPs. La ingeniería inversa completa es un lujo que rara vez cabe en el reloj del incidente.

? **¿Puedo confiar solo en una sandbox automática?** Son un gran punto de partida, pero el malware evasivo detecta sandboxes y ajusta su comportamiento. Complementa con análisis manual y correlación con tu telemetría.

? **¿Por qué priorizar TTPs sobre hashes?** Según la "pirámide del dolor", cambiar un hash o una IP es trivial para el atacante; cambiar sus TTPs es costoso. Detectar comportamiento resiste mejor la rotación de indicadores.

? **¿Es seguro usar servicios online de análisis?** Solo con muestras no confidenciales: al subirlas quedan accesibles a terceros. Para casos sensibles, usa una sandbox local (CAPEv2/Cuckoo) o análisis manual.

? **¿Qué diferencia al análisis de malware DFIR del *reversing* clásico?** El DFIR busca velocidad y utilidad operativa (contener el incidente); el *reversing* busca comprensión total del binario. Aquí el objetivo es actuar a tiempo.

## 🔗 Referencias

- Sikorski & Honig — *Practical Malware Analysis* (No Starch Press).
- SANS FOR610 *Reverse-Engineering Malware: Malware Analysis Tools and Techniques*.
- MITRE ATT&CK: <https://attack.mitre.org/>
- Documentación de `capa` (Mandiant/FLARE): <https://github.com/mandiant/capa>
- David Bianco — *The Pyramid of Pain*: <https://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html>

## **Siguiente clase**

---

Clase 327 - Ingeniería de detección avanzada y validación